

# Take-Home Project: Multi-Tenant Notification Platform

**Time budget:** 2–4 hours **Suggested submission window:** 5–7 business days from receipt

---

## Context

You'll be building a small but representative slice of a real product: a notification platform that ingests events from multiple customer organizations ("tenants"), applies their routing rules, and dispatches notifications on the appropriate channels.

The shape of the problem is familiar — what makes it interesting is that **every tenant must be safely isolated from every other tenant**, both in their data and in their ability to consume system resources.

This project is intentionally scoped so that completing it without AI assistance in the time budget would be very difficult. **You are expected to use AI tools** (Copilot, Claude, Cursor, etc.) as part of your normal workflow. We care that you understand, can defend, and can extend the code you submit — not that you typed every character yourself.

---

## The System

You will build an end-to-end service with four moving parts:

1. **Ingestion** — an HTTP endpoint that accepts events tagged with a `tenant_id`
2. **Routing** — a rule engine that determines, per tenant, which channels (if any) should receive a notification for a given event
3. **Dispatch** — the mechanism that actually sends the notification on the matched channel(s)
4. **Admin API** — endpoints for managing tenants and routing rules

Authentication is **out of scope**. Tenants identify themselves by sending `tenant_id` in the request body. This is not realistic for production, and that's fine — keep your focus on isolation, routing, and rate limiting.

---

## Functional Requirements

## Event Ingestion

- Expose an HTTP endpoint that accepts an event payload including `tenant_id`, an event type, and arbitrary additional fields.
- Validate the request and return appropriate error responses for malformed input or unknown tenants.
- Apply the requesting tenant's routing rules and dispatch notifications accordingly.

## Routing Rules

- Routing rules are **stored in the database** and are **tenant-scoped** — a rule belongs to exactly one tenant and is only applied to that tenant's events.
- The rule model itself is your design decision. At minimum it must let a tenant say "for events matching X, dispatch to channel(s) Y." How expressive the matching is, and how channels are represented, is up to you.
- Be prepared to defend your choice in the design doc and the interview.

## Dispatch

- Implement a clean dispatcher abstraction so that adding a new channel (email, Slack, webhook, SMS, etc.) does not require changes to the routing engine.
- The actual delivery mechanism is your call — real HTTP calls to a mock server, structured log lines, files on disk, in-memory capture, or pluggable concrete implementations. Whatever you pick, the **abstraction** is what we'll evaluate.

## Tenant Isolation

- Choose your isolation strategy (shared database with `tenant_id` column, schema-per-tenant, database-per-tenant, etc.) and defend it in the design doc.
- Whatever strategy you pick, **tenant A must not be able to read, modify, or affect tenant B's data or operations** under any code path exposed by your service.

## Rate Limiting

- Each tenant has its own rate limit. Both the algorithm (fixed window, sliding window, token bucket, leaky bucket, etc.) and the granularity (global per tenant, per channel, per event type, etc.) are your design decisions.
- Tenant A exhausting their limit must not degrade tenant B's experience.
- Document your choices and trade-offs in the design doc.

## Admin API

- CRUD endpoints for **tenants** (create, read, update, delete).
- CRUD endpoints for **routing rules**, scoped to a tenant.
- Standard error handling and validation.

---

# Deliverables

## 1. Runnable Service

- Submitted as a private Git repository (preferred) or a ZIP archive.
- **Any language, framework, and datastore** are fair game. Pick what lets you move fast and produce something you can defend.
- The team must be able to clone and start the service with **a single command** documented in the README. That command can be `docker compose up`, `make run`, `npm start`, `cargo run`, or whatever fits your stack — your call.

## 2. README

The README must include:

- Setup and run instructions (the single command, plus prerequisites).
- A way to **exercise the API** without reading the code first. Either a sample HTTP collection (Postman, Bruno, `.http` file, etc.) committed to the repo, or `curl` examples in the README covering the key flows (creating a tenant, creating a rule, ingesting an event).
- A brief description of architectural decisions (this can be a pointer to the design doc).
- Known limitations and what you would do differently with more time.
- Optionally, a note on which AI tools you used and where. This is not evaluated negatively — it signals self-awareness.

## 3. Design Document

A separate document (Markdown is fine) covering your architectural decisions and the trade-offs behind them. At minimum address:

- **Data model** — entities, relationships, where tenant scoping lives.
- **Isolation strategy** — what you chose, what you considered, why this one wins given the constraints.
- **Rate limiting** — algorithm, granularity, where state is stored, what happens when limits are exceeded.
- **Dispatcher abstraction** — the interface, how a new channel would be added.

Concise is better than exhaustive. We'd rather read 2–3 tight pages with real trade-offs than 10 pages of generated prose.

## 4. Tests (Encouraged, Not Required)

Tenant isolation tests are explicitly **encouraged but not required**. If you don't write automated tests, the README should walk us through manually demonstrating that tenant A cannot affect tenant B. If you do write tests, focus them where they earn their keep — isolation and routing are the natural targets.

---

## What We Evaluate

This project will be reviewed before your final interview, where we'll spend the bulk of our time walking through the code with you. We're specifically looking for:

- **Multi-tenancy thinking** — Does isolation hold under realistic failure modes? Did you think about the places it could leak?
  - **API design** — Are the endpoints coherent, well-shaped, and well-validated? Do the admin and ingestion surfaces feel like they belong to the same system?
  - **Abstraction quality** — Is the dispatcher genuinely extensible, or extensible-in-name-only?
  - **Trade-off reasoning** — When you made a call (isolation strategy, rate-limit algorithm, rule model), can you articulate what you traded away?
  - **Code you can defend** — Whatever you submit, you should be able to walk us through it line by line and explain why it's that way.
- 

## Time Budget Guidance

We expect this to take **2–4 hours of focused work** using AI assistance. If you find yourself well past that, stop and write up what you would have done in the design doc instead. We'd rather see a tight, defensible slice than a sprawling, half-finished system.

Good areas to defer if you're running long: extensive test coverage, polish on the admin API, fancy dispatcher implementations. Don't defer: tenant isolation, the rule model, rate limiting, the design doc.

---

## Submission

Send us the repo link (or ZIP) plus the design doc by the agreed deadline. We'll clone it, run it, and come to the final interview with 3–5 specific areas we'd like to dig into with you.

Good luck — we're looking forward to seeing what you build.